

MATLAB 官方教程

深度学习入门之旅

Examine predictions

Instructions are in the task pane to the left. Complete and submit each task one at a time.

This code loads in an image and imports GoogLeNet.

```
1  img = imread("file03.jpg");
2  imshow(img)
3  net = googlenet;
4  categorynames = net.Layers(end).ClassNames;
```

Task 1

```
5  [pred,scores] = classify(net,img);
```

Task 2

```
6  bar(scores)
```

```
1  img = imread("file03.jpg");
2  imshow(img)
3  net = googlenet;
4  categorynames = net.Layers(end).ClassNames;
```

Task 1

```
5  [pred,scores] = classify(net,img);
```

Task 2

```
6  bar(scores)
```

Task 3

```
7  highscores = scores > 0.01;
```

Task 4

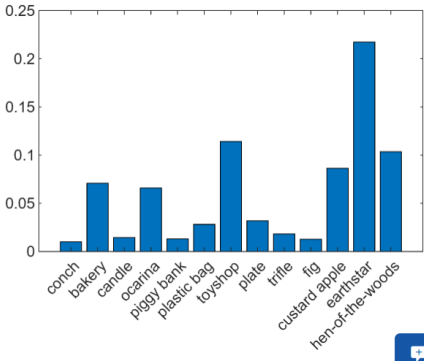
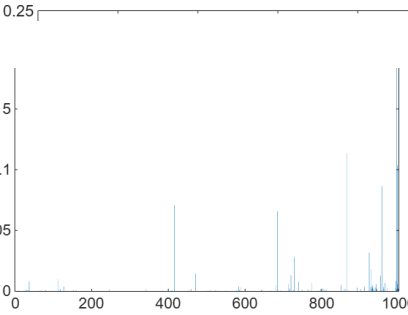
```
8  bar(scores(highscores))
```

Task 5

```
9  xticklabels(categorynames(highscores))
```

Further Practice

```
10
11
```



导入图像流

在一个文件夹中选取多幅图片进行识别，并附上标签

1

2

This code displays the images in the current folder and imports GoogLeNet.

```

1  ls *.jpg
2  net = googlenet;

```

3

4

Task 1

```

3  imds = imageDatastore("file*.jpg")
4

```

5

6

Task 2

```

5  fname = imds.Files
6

```

7

8

Task 3

```

7  img = readimage(imds,7)
8

```

9

10

Task 4

```

9  [preds,scores] = classify(net,imds)
10

```

file01.jpg file02.jpg file03.jpg file04.jpg file05.jpg file06.j

```

imds =
    ImageDatastore - 属性:

    Files: {
        '/users/mwa0000027732560/.training/.wc
        '/users/mwa0000027732560/.training/.wc
        '/users/mwa0000027732560/.training/.wc
        ... and 9 more
    }
    Folders: {
        '/users/mwa0000027732560/.training/.wc
    }
    AlternateFileSystemRoots: {}
    ReadSize: 1
    Labels: {}
    SupportedOutputFormats: ["png" "jpg" "jpeg" "tif"
    DefaultOutputFormat: "png"
    ReadFcn: @readDatastoreImage

fname = 12x1 cell
'./users/mwa0000027732560/.training/.work/file01.jpg'
'./users/mwa0000027732560/.training/.work/file02.jpg'
'./users/mwa0000027732560/.training/.work/file03.jpg'
'./users/mwa0000027732560/.training/.work/file04.jpg'
'./users/mwa0000027732560/.training/.work/file05.jpg'
'./users/mwa0000027732560/.training/.work/file06.jpg'
'./users/mwa0000027732560/.training/.work/file07.jpg'
'./users/mwa0000027732560/.training/.work/file08.jpg'
'./users/mwa0000027732560/.training/.work/file09.jpg'
'./users/mwa0000027732560/.training/.work/file10.jpg'

```

使用不同大小的图像进行识别

单个文件：

1

2

```

1  img = imread("file01.jpg");
2  imshow(img)

```

3

Task 1

```

3  sz = size(img)

```

4

5

6

Task 2

```

4  net = googlenet;
5  inlayer = net.Layers(1)
6  insz = inlayer.InputSize

```

7

8

Task 3

```

7  img = imresize(img,[224 224]);
8  imshow(img)

```

多个文件：使用增强的图像数据存储

resizeDatastore.mlx ×

Resize Images in a Datastore

Instructions are in the task pane to the left. Complete and submit each task one at a time.

This code displays the images in the current folder and imports GoogLeNet.

```

1  ls *.jpg
2  net = googlenet

```

Task 1

```

3  imds = imageDatastore("*.jpg")

```

Task 2

```

4  auds = augmentedImageDatastore([224 224],imds)
5

```

Task 3

```

6  preds = classify(net,auds)
7

```

```

Files: {
  '/users/mwa0000027732560/.training/.wc
  '/users/mwa0000027732560/.training/.wc
  '/users/mwa0000027732560/.training/.wc
  ... and 9 more
}
Folders: {
  '/users/mwa0000027732560/.training/.wc
}
AlternateFileSystemRoots: {}
ReadSize: 1
Labels: {}
SupportedOutputFormats: ["png" "jpg" "jpeg" "tif"
DefaultOutputFormat: "png"
ReadFcn: @readDatastoreImage

auds =
augmentedImageDatastore - 属性:

    NumObservations: 12
           Files: {12x1 cell}
AlternateFileSystemRoots: {}
    MiniBatchSize: 128
    DataAugmentation: 'none'
    ColorPreprocessing: 'none'
    OutputSize: [224 224]
    OutputSizeMode: 'resize'
    DispatchInBackground: 0

preds = 12x1 categorical
    seashore
    lakeside
    frying pan
    Shetland sheepd...
    toyshop

```

将一维的灰度图像转化为三维图像，颜色预处理以便神经网络

Preprocess Color Using a Datastore

Instructions are in the task pane to the left. Complete and submit each task one at a time.

This code displays the images in the current folder and imports GoogLeNet.

```

1  ls *.jpg
2  net = googlenet

```

This code creates an image datastore of these images.

```

3  imds = imageDatastore("file*.jpg")

```

Task 1

```

4  montage(imds)
5

```

Task 2

```

6  auds = augmentedImageDatastore([224 224],imds,"ColorPreprocessing","gray2rgb")
7

```

Task 3

```

8  net = googlenet
9  preds = classify(net,auds)

```

```

InputNames: {'data'}
OutputNames: {'output'}

imds =
ImageDatastore - 属性:

    Files: {
      '/users/mwa0000027732560/.training/.wc
      '/users/mwa0000027732560/.training/.wc
      '/users/mwa0000027732560/.training/.wc
      ... and 9 more
    }
    Folders: {
      '/users/mwa0000027732560/.training/.wc
    }
AlternateFileSystemRoots: {}
ReadSize: 1
Labels: {}
SupportedOutputFormats: ["png" "jpg" "jpeg" "tif"
DefaultOutputFormat: "png"
ReadFcn: @readDatastoreImage

```



将子文件夹中的图像进行分类:

使用 `imageDatastore` 中的第二个参数来实现

Create a Datastore Using Subfolders

Instructions are in the task pane to the left. Complete and submit each task one at a time.

This code imports GoogLeNet.

```
1 net = googlenet;
```

Task 1

```
2 flwrds = imageDatastore("Flowers","IncludeSubfolders",true)  
3
```

Task 2

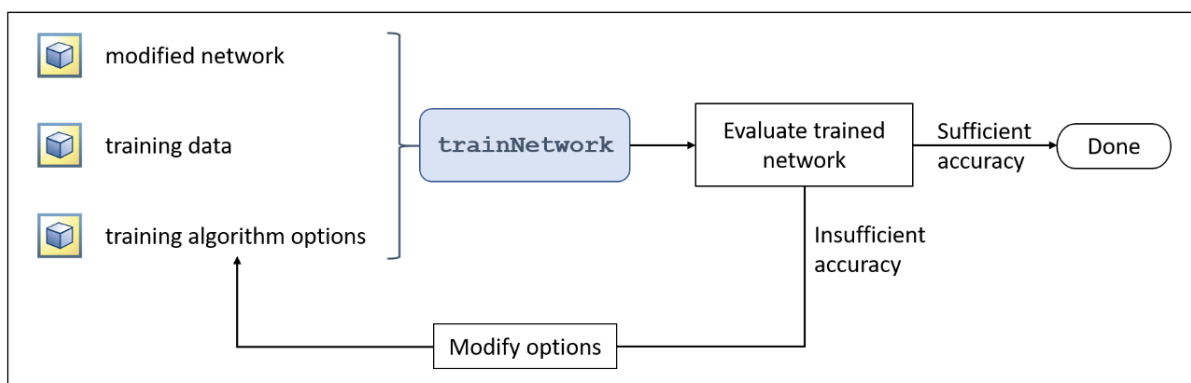
```
4 resizeds = augmentedImageDatastore([224 224],flwrds)  
5
```

Task 3

```
6 preds = classify(net,resizeds)
```

使用迁移学习进行对于网络的修改

迁移学习的典型 workflow



要执行迁移学习，需要具备三个要素：

1. 一个表示网络架构的层数组。对于迁移学习，该数组是通过修改现有的网络（如 GoogLeNet）来创建的。
2. 具有已知标签的图像，用作训练数据。它通常以数据存储形式提供。
3. 一个包含用于控制训练算法行为的选项的变量。

这三个要素作为输入提供给 `trainNetwork` 函数，该函数返回经过训练的网络作为输出。

您需要测试新训练的网络的性能。如果性能不够理想，通常您应该尝试调整一些训练选项并重新训练。

导入变量时将文件名一起导入

Label Images in a Datastore

Instructions are in the task pane to the left. Complete and submit each task one at a time.

This code creates a datastore of flower images.

```
1 load pathToImages
2 flwrds = imageDatastore(pathToImages,"IncludeSubfolders",true);
3 flowerNames = flwrds.Labels
```

Task 1

```
4 flwrds = imageDatastore(pathToImages,"IncludeSubfolders",true,"LabelSource","folderNames");
5
```

Task 2

```
6 flowerNames = flwrds.Labels
7
```

分割数据集

可选随机，也可以指定数量：

This code creates a datastore of flower images.

```
1 load pathToImages
2 flwrds = imageDatastore(pathToImages,"IncludeSubfolders",true,"LabelSource","folderNames")
```

Task 1

```
3 [flwrTrain,flwrTest] = splitEachLabel(flwrds,0.6)
```

Task 2

```
4 [flwrTrain,flwrTest] = splitEachLabel(flwrds,0.8,"randomized")
```

Task 3

```
5 [flwrTrain,flwrTest] = splitEachLabel(flwrds,500)
```

首先获取数据存储，根据实际情况选择是否需要增强数据。

使用神经网络设计器进行可视化设计

设置训练模式

设置包含训练选项的变量，如 *求解策略* 和 *初始学习率*

1	Task 1 <pre>opts = trainingOptions("sgdm")</pre>
2	Task 2 <pre>opts = trainingOptions("sgdm","InitialLearnRate",0.001)</pre>

典型的分类脚本

	Transfer Learning Example Script Instructions are in the task pane to the left. There are no graded tasks.
1 2 3 4 5 6	Get training images and process them using datastores. <pre>load pathToImages flower_ds = imageDatastore(pathToImages,"IncludeSubfolders",true,"LabelSource","foldernames"); [trainImgs,testImgs] = splitEachLabel(flower_ds,0.6); resizeTrainImgs = augmentedImageDatastore([224 224],trainImgs); resizeTestImgs = augmentedImageDatastore([224 224],testImgs); numClasses = numel(categories(flower_ds.Labels));</pre>
7	Create a network by modifying GoogLeNet. First, launch the Deep Network Designer. <pre>deepNetworkDesigner</pre> Then replace the fullyConnectedLayer (Convolution and Fully Connected) and the classificationLayer (Output). <pre>load untrainedNetwork</pre>
8	Set training algorithm options. The default number of epochs is 30, which means the entire training data set goes through the network 30 times. To reduce training time on the CPU, the number of epochs is limited to 1. The verbose frequency is also decreased so that you see updates more frequently. <pre>opts = trainingOptions("sgdm","InitialLearnRate",0.001,"MaxEpochs",1,"VerboseFrequency",2);</pre>
9	Perform training by passing all three inputs to the trainNetwork function. The network training should take about two minutes on a CPU. <pre>[flowerNet,info] = trainNetwork(resizeTrainImgs,lgraph,opts)</pre>

评估网络的性能

使用混淆矩阵命令评判结果的正确与否：

```

1  %%% testImages
2  load trainedFlowerNetwork
3  flwrds = imageDatastore(pathToImages,"IncludeSubfolders",true,"LabelSource","foldernames");
4  [trainImgs,testImgs] = splitEachLabel(flwrds,0.99);
5  resizeTestImgs = augmentedImageDatastore([224 224],testImgs);
6  flwrPreds = classify(flowernet,resizeTestImgs);

```

Task 1

```

7  flwrActual = testImgs.Labels
8

```

Task 2

```

9  numCorrect = nnz(flwrActual == flwrPreds)
10

```

Task 3

```

11  fracCorrect = numCorrect / numel(flwrPreds)
12

```

Task 4

```

13  confusionchart(flwrActual,flwrPreds)
14

```

```

flwrActual = 34x1 categorical
    daisy
    daisy
    daisy
    daisy
    daisy
    daisy
    dandelion
    dandelion
    dandelion
    dandelion
    ...

```

numCorrect = 31

fracCorrect = 0.9118

daisy	6
dandelion	1

使用网络预测测试集的结果，与文件夹的正确分类进行比较

您已确定 `flowerNet` 网络对测试数据的分类准确率为 91% (31/34)。请注意，其中一个误分类预测为雏菊，而它实际上是蒲公英。

要研究误分类的图像，您可以找到哪些文件包含误分类的图像并查看这些图像。例如，以下代码显示第二个误分类的图像：

```

idxWrong = find(flwrPreds ~= flwrActual)
idx = idxWrong(1)
imshow(readimage(testImgs,idx))
title(testImgs.Labels(idx))

```

迁移学习函数摘要

创建网络

函数	说明
deepNetworkDesigner	启动深度网络设计器
googlenet	加载预训练网络“GoogLeNet”
支持的网络	查看可用的预训练网络列表
fullyConnectedLayer	创建新的全连接网络层
classificationLayer	为分类网络创建新输出层

获取训练图像

函数	说明
imageDatastore	创建引用图像文件的数据存储
augmentedImageDatastore	预处理图像文件集合
splitEachLabel	将数据存储划分为多个数据存储

设置训练算法选项

函数	说明
trainingOptions	创建包含训练算法选项的变量

设置训练算法选项

函数	说明
trainingOptions	创建包含训练算法选项的变量

执行训练

函数	说明
trainNetwork	执行训练

使用经过训练的网络执行分类

函数	说明
classify	获取经过训练的网络对输入图像的分类

评估经过训练的网络

函数	说明
nnz	统计数组中的非零元素
confusionchart	计算混淆矩阵